

QBF Extended Failed Literal Detection

SAM WOOD, University of California, Santa Cruz

Recent advances in Quantified Boolean Formula preprocessors allow quick detection of some false formulas using incomplete proof systems. In cases where a formula is not solved, preprocessors can generate a satisfiably equivalent formula with additional learned clauses that may help solvers efficiently find an unsatisfiable assignment. This paper details a method of learning unit clauses and binary clauses by extending boolean constraint propagation, using a method called Extended Failed Literal detection (EFL). The QxBF solver is modified to include EFL, and performance measurements against the QBFEVAL'10 benchmark suite are detailed. EFL is simple to implement, solves more formulas, and produces formulas that are faster to analyze by QBF solvers.

General Terms: Design, Algorithms, Performance

Additional Key Words and Phrases: Quantified boolean formula, failed literal detection, boolean constraint propagation

ACM Reference Format:

Wood, S. 2011. QBF Extended Failed Literal Detection ACM Trans. Embedd. Comput. Syst. V, N, Article A (January YYYY), 7 pages.

DOI = 10.1145/0000000.0000000 <http://doi.acm.org/10.1145/0000000.0000000>

1. INTRODUCTION

Quantified Boolean Formula (QBF) refers to a formal language that extends propositional logic by allowing variables to have a single existential ($\exists$) or universal ($\forall$) quantifier. QBF are typically used as a formalism to describe mathematical theorems, while QBF solvers apply automated reasoning to verify the veracity of the theorem. Such reasoning applies a proof system, typically guided by heuristics, to derive additional clauses which are logically implied by the original formula, and can be soundly added to the formula.

Variables that do not have a corresponding quantifier are labeled as *free*, whereas a formula without any *free* variables is known as *closed*. The QBF decision problem is the canonical PSPACE-complete problem and asks whether a *closed* QBF is *True* or *False*. It is becoming evident that a solver simply providing a yes or no answer to this question is not enough: ideally a solver should output the steps of the derivation. Indeed, the EFL method detailed in this paper uses sound (*satisfiably equivalent*, both formulas either have models or both do not have any models) reasoning to derive unit clauses, and is amenable to logging the steps of the derivation.

QBF preprocessors are programs which take as input a *closed* QBF in *prenex conjunctive normal form*¹ (PCNF) and output another PCNF QBF that is satisfiably equiv-

¹A QBF is in PCNF if and only if all of the quantifiers occur in the prefix of the formula and the *matrix* contains all of the clauses in conjunctive normal form.

Author's address: S. Wood, Computer Engineering Department, University of California, Santa Cruz.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.

© YYYY ACM 1539-9087/YYYY/01-ARTA \$10.00

DOI 10.1145/0000000.0000000 <http://doi.acm.org/10.1145/0000000.0000000>

alent to the original, in polynomial time using an incomplete proof system. The goal of the preprocessor is to (quickly) output a formula that can be solved by the solver (which uses complete reasoning) in less time than the untransformed formula. In some cases, the original formula is simple enough that its decision problem can be solved by the preprocessor alone, in which case the preprocessor should output a trivial QBF instance. Note that any QBF can be transformed to a satisfiably equivalent PCNF (which may contain more variables) in linear time.

1.1. Boolean Constraint Propagation

One technique used by preprocessors such as QxBF [Lonsing and Biere 2011] is failed literal detection. In this case, an existential literal is assigned *True* (an assumption) at *decision level 1*, and *boolean constraint propagation* (QBCP) is applied. In other words, if the assumption turns some clause in the formula's matrix into a unit clause, and the remaining literal in this clause is existentially quantified, then this literal is also assigned *True* at *decision level 1*, and the process is repeated. If the empty clause is derived then a new unit clause is added to the original formula containing the negation of the assumed literal (by assigning the negated literal as *True* at *decision level 0*). Once QBCP terminates, backtracking occurs and the assignments at *decision level 1* are reverted. The goal of QBCP is to derive an empty clause and learn a new unit clause. Practically speaking, by this method's nature of adding additional constraints to the formula, this method is mainly used to prove that a formula is *False*.

In addition to the unit clause propagation, QBCP employs three methods for eliminating variables and clauses: universal reduction, the pure existential literal rule and the pure universal literal rule.

Definition 1.1 (Universal Reduction). Universal reduction is a sound transformation (preserves satisfiability) which states that all universally quantified literals that are at a scope depth greater than the scope depth of the deepest existentially quantified literal within the clause can be removed.

Definition 1.2 (Pure Existential Rule). The pure existential rule states that if all of the occurrences of an existential variable in a formula have the same sign, then all of the clauses containing that variable can be removed. To be sure, the pure existential literal is a *safe heuristic* in that it is sound when used to derive the empty clause, but generally it does not produce satisfiability equivalent formulas.

Definition 1.3 (Pure Universal Rule). The pure universal literal rule states that if all of the occurrences of universal variable in a formula have the same sign, then this literal can be deleted from all of the clauses. It is in general a sound transformation.

During QBCP, QxBF employs the notion of an abstracted formula with the goal of deriving more unit clauses. When assigning a variable at *decision level 1*, all variables with quantifier scope depth less than the assigned variable are treated as existential. The new quantifier of the variables that were universally quantified in the original formula is denoted as an *abstracted existential* quantifier.

1.2. Extended Failed Literal Detection

Extended failed literal (EFL) detection is a technique used in conjunction with QBCP that allows learning of additional unit clauses. Specifically, after assuming a variable to be *True* we build a set of the implied existential literals, say A . If the empty clause was not derived, then we build another set of the implied existential literals, say B after making the opposite assumption. If the empty clause is also not derived in this case, we then intersect $A \cap B$ and add the intersected literals as unit clauses to the original formula (assign them to be *True* at decision level 0). Without proof, we note

that this method is sound when used in conjunction with universal reduction and the universal pure literal rule. However, EFL is incompatible with the pure existential rule; indeed, there exist satisfiable formula that become unsatisfiable (*False*, inconsistent) after applying EFL with the pure existential rule. Also, EFL is sound when including literals with *abstracted existential* quantifiers in sets A and B .

EFL can be further extended to discover binary clauses that represent equivalent or opposite variables. Two variables x and y are considered *equivalent* in ψ if the clauses $\neg x \vee y$ and $\neg y \vee x$ are logically implied by formula ψ . In other words, they have the same sign in ψ . Two variables x and y are considered *opposite* in ψ if the clauses $x \vee y$ and $\neg y \vee \neg x$ are logically implied by formula ψ . In other words, they have opposite sign in ψ . Binary clause learning naturally follows from EFL: when performing the set intersection $A \cap B$ after assuming x and $\neg x$, if two literals have the same variable, say y , and y is positive in A and negative in B , then x and y are equivalent (since we assumed x was positive when constructing A , and negative when constructing B). Analogously, if A contains some positive literal with variable y , and B contains a negative literal with variable y , then x and y are opposites.

2. MODIFIED QXBF IMPLEMENTATION

QxBF1.1² is the latest version of the QxBF preprocessor. QxBF uses three mechanisms to learn unit clauses: abstracted based QBCP (ABS), SAT, and Q-resolution (QRES). ABS and QRES are incompatible, but SAT is compatible with the other two. QxBF allows users to specify total time limits and propagation limits to bound the amount of time spent in each method. Both methods (ABS+SAT or QRES+SAT) take turns in rounds until either the formula is proven UNSAT or a limit is exceeded. To be sure, *cputime* limits external to the program must also be set when evaluating benchmarks since the internal limit checks cannot be very precise without incurring performance degradation.

Practically speaking, QxBF is ideally suited for prototyping EFL since it is relatively small (3385 lines of code, including comments) and already contains most of the pre-requisite mechanisms and data structures (e.g. propagating unit clauses and storing implied unit clauses). The EFL patch to QxBF1.1 is 580 lines, including comments and compiler flags to switch between the different EFL variants (see Section 3). Beyond a typical amount of debugging and familiarization with QxBF, EFL was relatively simple to implement; it was completed and debugged within 4 weeks. To be sure, the pure existential literal rule had to be disabled prior to adding EFL.

Since we are mainly interested in the effectiveness of EFL (which is an extension to ABS), we have not modified or examined the QRES code.

2.1. Unit Clauses

Prior to any modifications, QxBF already had a mechanism for deriving a list of learned unit clauses after making an assumption. Additionally, QxBF had a global VAR[] array that contains metadata for each variable in the formula. VAR[] is updated during QBCP in order to backtrack and revert the changes caused by assuming a literal to be *True* or *False*.

Algorithm 1 details the pseudocode for the QxBF implementation. We augmented the data structure in the VAR[] array to include two single bit fields: *intersect* and *intersect.phase*. *intersect* is marked to *True* if the particular variable was assigned at decision level 1 after making an assumption, and remains *False* otherwise. *intersect.phase* records the phase of the assignment which made the derived unit clause

²<http://fmv.jku.at/qxbf/>, June 2011

True. Since every variable in the formula has a `VAR[]` entry, we were careful to be conservative in adding fields to this data structure. To be sure, the data structure in the `VAR[]` array already had several single bit fields and had space remaining such that the additional fields did not increase the struct size.

Code was added to QxBF after making the positive assumption for a variable and prior to backtracking. Specifically, we inspect the derived unit clauses (assigned literals at decision level 1) and mark their respective *intersect_phase* and *intersect* bits if they are existentially quantified (including abstract existentials). During marking we also add the variable to a list (*mark_list*) in order to efficiently unmark it later. Alternatively, we could avoid using a *mark_list* and iterate across all of the variables setting them as unmarked—we avoided this method since it may harm cache performance.

One key difference between Algorithm 1 and the actual implementation is that if the empty clause is derived after either assumption (lines 2 and 11) then the code exits without performing any further intersections. In this case, QxBF applies the necessary assignment and assigns the variables for any derived unit clauses to be *True* at decision level 0.

After marking the variables the negative assumption is made and QBCP propagation is again carried out. However, instead of marking the variables of the implied unit clauses we now check if these variables were previously assigned during the positive assumption. If a variable was assigned and it has the same phase for the positive assumption as for the negative assumption then this variable is in the intersection and we add the derived unit clause to the *assigned_list*. After computing the intersection, we backtrack and unmark all the marked variables set prior to the negative assumption. Note that due to QxBF's implementation, we cannot easily apply assignments at decision level 0 after applying assignments at decision level 1 (and prior to backtracking); therefore we must store the assignments found by EFL in the *assigned_list*, backtrack, and then apply these assignments at decision level 0.

Observe that this algorithm only uses additional space when storing variables in the *mark_list* and *assign_list*—both are of size $O(n)$, where n is the number of variables. Similarly, the additional time spent looping over the *derived_list* lists is $O(n)$. Hence, we can conclude that the algorithm does not introduce an asymptotically greater time or space complexity.

2.2. Equivalent Variables

Algorithm 2 is an extension to Algorithm 1 that learns necessary binary clauses to encode derived equivalent or opposite variables. Due to QxBF's limited scope as a pre-processor, there is no mechanism for efficiently marking a variable as being equivalent to another variable. In order to quickly work around this pitfall, we have implemented a proof-of-concept of Algorithm 2 that simply outputs all derived binary clauses to a temporary scratch file as soon as they are derived. Later we append this file to the list of clauses once QxBF is done processing (assuming the formula was not solved).

This solution is inadequate: mainly, because the learned binary clauses do not participate in any further propagation and only contribute to processing that may occur on the output file. Also, this method does not check for duplicate binary clauses, therefore many of the learned binary clauses are redundant. Indeed, redundant binary clauses are common since a variable may be propagated multiple times, each time rediscovering the binary clause. Additionally, the same binary clause can be learned when assuming one of the other variables in the same equivalence class is *True* or *False* (e.g. if assuming α and $\neg\alpha$ discovers $\alpha = \beta$, then assuming β and $\neg\beta$ may rediscover $\alpha = \beta$). Lastly, we do not check if the learned binary clauses are already in the formula.

ALGORITHM 1: Necessary unit clauses

Input: A QBF ψ and a variable identifier $\alpha \in \mathbb{N}$ to undergo boolean constraint propagation.
Output: A list of necessary unit clauses $\{\phi_i : \text{where } \phi_i = \{\beta_i\} \text{ or } \phi_i = \{\neg\beta_i\}, \text{ for variable identifier } \beta_i \in \mathbb{N}, \text{ and } \phi_i \text{ is true in } \psi\}$.
Data: A global array `VARs[]` contains metadata for each variable identifier $\beta_i \in \mathbb{N}$, and `assume_positive/negative_qbcp()` modifies `VARs[\beta_i].derived_phase` for each derived literal with variable identifier β_i

```

1 mark_List = []; assign_List = [];
  /* assume  $\alpha$  in  $\psi$ , see what unit clauses are derived */
2 derived_List = assume_positive_qbcp( $\psi$ ,  $\alpha$ );
3 foreach  $\beta_i \in \text{derived\_List}$  do
  /* only add (including abstracted) existentially quantified variables */
4   if IS_ABSTRACT_EXIST( $\text{var}$ ) then
5     VARs[\beta_i].intersect  $\leftarrow$  True;
6     VARs[\beta_i].intersect_phase  $\leftarrow$  VARs[\beta_i].derived_phase;
7     append(mark_List,  $\beta_i$ );
8   end
9 end
  /* revert the assignments made by bcp when assuming  $\alpha$  */
10 backtrack_qbcp( $\psi$ , derived_List);
  /* assume  $\neg\alpha$  in  $\psi$ , see what unit clauses are derived */
11 derived_List = assume_negative_qbcp( $\psi$ ,  $\alpha$ );
12 foreach  $\beta_i \in \text{derived\_List}$  do
13   if VARs[\beta_i].intersect == True then
14     if VARs[\beta_i].derived_phase == VARs[\beta_i].intersect_phase then
15       /* variable derived when assuming both  $\alpha$  and  $\neg\alpha$  */
16       if VARs[\beta_i].derived_phase == True then
17         append(assign_List,  $\{\beta_i\}$ );
18       else
19         append(assign_List,  $\{\neg\beta_i\}$ );
20       end
21     else
22       /* Equivalent literal code (Algorithm 2) goes here */
23     end
24   end
25 end
  /* revert the assignments made by bcp when assuming  $\neg\alpha$  */
26 backtrack_bcp( $\psi$ , derived_List);
  /* unmark previously marked variables */
27 foreach  $\beta_i \in \text{mark\_List}$  do
28   VARs[\beta_i].intersect  $\leftarrow$  False;
29 end
30 return assign_List;

```

3. PERFORMANCE EVALUATION

All experiments were conducted on the UCSC campusrocks computer cluster donated by craigslist³. Each compute node had dual 64-bit 2.6GHz AMD Opteron 252 processors with 16GB of RAM, using CentOS 5.3. The *filelimit* was set to 128MB, and *vmemoryuse* was set to 9GB. Exceeding these limits was considered a time out.

We modified QxBF1.1 to include the following variations of EFL in order to understand what methods are effective:

³<https://support.soe.ucsc.edu/craigslist/>, June 2011

ALGORITHM 2: Necessary binary clauses**Input:** Global VAR[] array, *assign_list* of derived clauses, and variable identifiers $\alpha, \beta_i \in \mathbb{N}$.**Output:** *assign_list* is modified to include derived binary clauses.**Data:** This code is meant to be inserted in line 20 of Algorithm 1.

```

1 if VARS[ $\beta_i$ ].intersect_phase == True then
    /*  $\alpha$  and  $\beta_i$  are equivalent */
2   append(assign_list, { $\neg\alpha, \beta_i$ });
3   append(assign_list, { $\neg\beta_i, \alpha$ });
4 else
    /*  $\alpha$  and  $\beta_i$  are opposites */
5   append(assign_list, { $\neg\alpha, \neg\beta_i$ });
6   append(assign_list, { $\beta_i, \alpha$ });
7 end

```

Table I.

QBF EVAL10: 568 formulae					
Preprocessing	Solver	Solved (Preproc.)	Time (Preproc.)	SAT	UNSAT
sQueuezeBF	DepQBF	421 (39)	502.3 (69.4)	195	226
sQueuezeBF + (ABS + SAT)		378 (64)	677.1 (86.3)	171	207
sQueuezeBF + (EFL + SAT)		424 (66)	488.5 (82.5)	197	227
sQueuezeBF + (EFL-abs + SAT)		424 (66)	492.9 (82.5)	197	227
sQueuezeBF + (EFL-nopure + SAT)		423 (64)	493.2 (81.8)	196	227
sQueuezeBF + (EFL-equiv + SAT)		422 (66)	492.3 (82.7)	195	227
ABS+SAT	DepQBF	377 (51)	674.5 (85.7)	170	207
EFL+SAT		382 (51)	673.4 (84.8)	173	209
EFL-abs+SAT		379 (51)	666.6 (84.8)	172	207
EFL-nopure+SAT		379 (51)	678.4 (83.2)	172	207
EFL-equiv+SAT		378 (51)	673.6 (84.3)	171	207
none		368 (0)	675.8 (0)	164	204

Note: Solver performance with time limited preprocessing. As in [Lonsing and Biere 2011], times are average total run times *including* preprocessing and time outs, while average preprocessing times are in parenthesis. Additionally, the number solved in parenthesis indicates the number of benchmarks solved by preprocessing alone. “None” indicates no preprocessing, “SAT” indicates the unmodified SAT-based processing in QxBF1.1, “ABS” indicates the unmodified abstraction-based FL in QxBF1.1. sQueuezeBF+. . . indicates that the sQueuezeBF processor was applied to the benchmarks and then the remaining time of the unsolved formulas were spent on the subsequent preprocessor and solver.

EFL. EFL with the pure universal literal rule, but without abstracted existentially quantified literals in the intersection and without binary clause learning.

EFL-abs. Same as *EFL*, but also includes abstracted existentially quantified literals in the intersection.

EFL-nopure. Same as *EFL-abs*, but without the pure universal literal rule.

EFL-equiv. Same as *EFL-abs*, but also includes binary clause learning.

Table I provides a comparison of the different FL approaches when used with DepQBF0.1 [Lonsing and Biere 2010], a solver based on search. Additionally, we compared EFL with sQueuezeBF, the preprocessor for Qube7.2 [Giunchiglia et al. 2010]. For these tests, we allowed a maximum of 160 seconds for preprocessing with any of the EFL variants (80 seconds SAT and 80 seconds BCP)⁴. Additionally, we set a 1800 second time limit for the combination of preprocessing and solving. The number of SAT and BCP propagations remained set to default. Since sQueuezeBF does not support resource limits, sQueueze ran until completion or a time out, while DepQBF used the

⁴Due to the inaccuracy in the internal counters, we also set a hard *cpulimit* of 200 seconds.

remaining time (if any). All of the benchmarks are from QBFEVAL'10 [Peschiera et al. 2010]. With these parameters, sQueue alone solved 39 formula and timed out on 12.

A key conclusion from these results is that unlike QxBF which is incompatible with sQueueBF, QxBF with the EFL extensions is compatible. Indeed, when comparing sQueueBF with EFL, three additional tests were solved using 13.8 fewer seconds. To be sure, EFL is not competitive with sQueueBF: EFL solves 39 fewer benchmarks in 172.2 more seconds. Also, for these cases the extensions to EFL did not lead to more benchmarks solved although they did lead to more benchmarks solved by preprocessing when used with sQueueBF.

4. CONCLUSIONS

For the QBFEVAL'10 benchmark suite, Extended Failed Literal detection proved to be an effective method of preprocessing when used in combination with sQueueBF, the current state of the art. Indeed, sQueueBF used with EFL and the DepQBF solver leads to more benchmarks solved in less time. EFL and its variants were simple to implement by modifying the QxBF preprocessor. It remains as further reasearch to explore whether EFL's variations can provide additional gains to QBF preprocessing and solving.

ACKNOWLEDGMENTS

The author would like to thank Prof. Van Gelder at the University of California, Santa Cruz, for his in-depth guidance in the theory, implementation and analysis of extended failed literal detection.

REFERENCES

- GIUNCHIGLIA, E., MARIN, P., AND NARIZZANO, M. 2010. sQueueBF: An effective preprocessor for qbfs based on equivalence reasoning. In *Theory and Applications of Satisfiability Testing SAT 2010*, O. Strichman and S. Szeider, Eds. Vol. 6175. 85–98.
- LONSING, F. AND BIERE, A. 2010. DepQBF: A dependency-aware qbf solver (system description). In *JSAT*. 7:71–76.
- LONSING, F. AND BIERE, A. 2011. Failed literal detection for QBF. In *Proceedings of the 14th International Conference on Theory and Applications of Satisfiability Testing (SAT'11)*.
- PESCHIERA, C., PULINA, L., TACCHELLA, A., BUBECK, U., KULLMANN, O., AND LYNCE, I. 2010. The seventh QBF solvers evaluation (QBFEVAL10). In *Theory and Applications of Satisfiability Testing SAT 2010*, O. Strichman and S. Szeider, Eds. Vol. 6175. 237–250.